30 SQL for Data Science Interview Questions & Answers

JOINs | Window Functions | CTEs | Cohort Analysis | A/B Testing | Feature Engineering

Real Data Science Scenarios | Asked at FAANG | 2026

SQL Fundamentals	**JOINs & Relationships**	**Aggregations & GROUP BY**
Window Functions	**Subqueries & CTEs**	**Data Cleaning**
	Advanced SQL for ML	

AmanAI Lab

amanailab.com | YouTube: @AmanAILab

FREE — github.com/amanailab/Production-level-Generative-AI-Interview-Questions

Table of Contents

■ SQL Fundamentals

Q1. What is the difference between WHERE and HAVING in SQL?

Q2. What is the difference between DELETE, TRUNCATE, and DROP?

Q3. Explain the SQL execution order. Why does it matter for Data Scientists?

Q4. What is the difference between UNION and UNION ALL?

■ JOINs & Relationships

Q5. Explain all types of JOINs with real Data Science examples.

Q6. How do you find duplicate records in a table using SQL?

Q7. What is a self-join and when do you use it in Data Science?

Q8. How do you handle many-to-many relationships in SQL for Data Science?

■ Aggregations & GROUP BY

Q9. How do you calculate running totals and moving averages in SQL?

Q10. What is ROLLUP and CUBE in SQL and how do you use them for data analysis?

Q11. How do you pivot data from rows to columns in SQL?

■ Window Functions

Q12. Explain window functions in SQL. What makes them different from GROUP BY?

Q13. What is the difference between ROW_NUMBER, RANK, and DENSE_RANK?

Q14. How do you use LAG and LEAD functions for time series analysis?

Q15. How do you calculate percentiles and quantiles in SQL?

■ Subqueries & CTEs

Q16. What are CTEs (Common Table Expressions) and when do you use them over subqueries?

Q17. What are correlated subqueries and when do you use them?

Q18. How do you write recursive CTEs for hierarchical data?

■ Data Cleaning & Transformation

Q19. How do you handle NULL values in SQL for Data Science?

Q20. How do you write SQL for cohort analysis in Data Science?

Q21. How do you calculate session durations and user journeys in SQL?

Q22. How do you detect and handle outliers using SQL?

■ Advanced SQL for ML & Analytics

Q23. How do you write SQL for funnel analysis?

Q24. How do you write SQL for A/B test analysis?

Q25. How do you write SQL to calculate customer lifetime value (CLV)?

Q26. How do you optimize slow SQL queries in production?

Q27. How do you use SQL for feature engineering for ML models?

Q28. How do you write SQL to analyze time series data for forecasting?

Q29. How do you write SQL queries for geospatial and JSON data in modern data warehouses?

SQL Fundamentals

Q1 What is the difference between WHERE and HAVING in SQL?

Answer:

WHERE filters rows BEFORE grouping — it works on individual rows from the raw table. **HAVING** filters AFTER grouping — it works on aggregated values. Rule: if you need to filter on an aggregate function (COUNT, SUM, AVG, MAX, MIN), you must use HAVING. If filtering on raw column values, use WHERE. You can use both in the same query — WHERE filters individual rows first, then GROUP BY aggregates, then HAVING filters the groups.

■ **SQL Example:**

```
-- Find departments with more than 5 employees earning above $50K
SELECT department, COUNT(*) as emp_count, AVG(salary) as avg_salary
FROM employees
WHERE salary > 50000 -- Filter rows BEFORE grouping
GROUP BY department
HAVING COUNT(*) > 5; -- Filter groups AFTER aggregation

-- WRONG (can't use COUNT in WHERE):
SELECT department, COUNT(*)
FROM employees
WHERE COUNT(*) > 5 -- ERROR! COUNT not allowed in WHERE
GROUP BY department;
```

■ *Interview Tip: Say 'WHERE filters rows, HAVING filters groups. WHERE runs before GROUP BY, HAVING runs after. If I need to filter on COUNT() or SUM(), I must use HAVING.' Then give a practical example — this shows you understand the SQL execution order.*

Q2 What is the difference between DELETE, TRUNCATE, and DROP?

Answer:

DELETE: Removes specific rows based on WHERE condition. Logged operation — can be rolled back. Slow for large tables (row by row). Table structure remains. **TRUNCATE**: Removes ALL rows from a table. Minimally logged — very fast. Cannot be rolled back in most databases. Resets auto-increment counters. Table structure remains. **DROP**: Removes the entire table including structure, data, indexes, constraints, and triggers. Cannot be undone (in most cases). The table no longer exists. Memory tip: DELETE = careful surgeon (selective), TRUNCATE = clear the room (all rows, fast), DROP = demolish the building (table gone).

■ **SQL Example:**

```
-- DELETE: remove specific rows (can rollback)
DELETE FROM orders WHERE status = 'cancelled' AND order_date < '2025-01-01';
-- Affects: only cancelled orders before 2025
-- Can: WHERE clause, triggers fire, can rollback

-- TRUNCATE: remove all rows instantly
TRUNCATE TABLE temp_staging_data;
-- Affects: ALL rows removed instantly
-- Can: no WHERE clause, no triggers, resets sequences
-- Speed: 100x faster than DELETE for full table clear

-- DROP: remove the table entirely
DROP TABLE old_backup_table;
-- Affects: table, data, indexes, constraints ALL gone
-- Cannot undo (without backup)

-- Data Science use case:
-- Truncate staging tables before loading new data
-- Delete specific records for GDPR compliance
-- Drop temporary tables after ETL pipeline
```

■ *Interview Tip: Mention the rollback difference. 'DELETE is transactional and can be rolled back. TRUNCATE cannot be rolled back in most databases. This matters for data pipelines — if a TRUNCATE runs and the subsequent INSERT fails, you've lost your data.' Shows production awareness.*

Q3 Explain the SQL execution order. Why does it matter for Data Scientists?

Answer:

SQL queries do NOT execute in the order you write them. The actual execution order is: (1) **FROM** — identify source tables. (2) **JOIN** — combine tables. (3) **WHERE** — filter rows. (4) **GROUP BY** — aggregate into groups. (5) **HAVING** — filter groups. (6) **SELECT** — choose columns. (7) **DISTINCT** — remove duplicates. (8) **ORDER BY** — sort results. (9) **LIMIT/OFFSET** — restrict output. Why it matters: (a) You CANNOT use SELECT aliases in WHERE (alias doesn't exist yet at WHERE stage). (b) You CAN use SELECT aliases in ORDER BY (ORDER BY runs after SELECT). (c) Aggregate functions only available after GROUP BY stage.

■ **SQL Example:**

```
-- WRONG: using SELECT alias in WHERE (alias not yet defined)
SELECT salary * 1.2 AS adjusted_salary
FROM employees
WHERE adjusted_salary > 60000; -- ERROR: unknown column

-- RIGHT: use the expression directly in WHERE
SELECT salary * 1.2 AS adjusted_salary
FROM employees
WHERE salary * 1.2 > 60000; -- Works!

-- OR use a subquery/CTE:
WITH adjusted AS (
SELECT salary * 1.2 AS adjusted_salary FROM employees
)
SELECT * FROM adjusted WHERE adjusted_salary > 60000;

-- RIGHT: alias CAN be used in ORDER BY (runs after SELECT)
SELECT salary * 1.2 AS adjusted_salary
FROM employees
ORDER BY adjusted_salary DESC; -- Works! ORDER BY runs after SELECT
```

■ *Interview Tip: 'FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. This order explains why you can use SELECT aliases in ORDER BY but not WHERE.' Saying the exact order from memory immediately impresses interviewers.*

Q4 What is the difference between UNION and UNION ALL?

Answer:

UNION: Combines results of two queries and removes duplicates. Slower because it must sort/compare all rows to find duplicates. **UNION ALL**: Combines results of two queries and KEEPS all duplicates. Faster because no deduplication step. Both require: same number of columns, compatible data types in each position. When to use each: Use UNION ALL when you know data won't have duplicates (different time periods, different sources) or when you want all records. Use UNION only when you specifically need deduplication. In data science: UNION ALL is almost always preferred for performance in ETL pipelines.

■ **SQL Example:**

```
-- UNION: removes duplicates (slower)
SELECT user_id, 'web' as source FROM web_sessions
UNION
SELECT user_id, 'mobile' as source FROM mobile_sessions;
-- If same user_id+source appears in both tables, shown once

-- UNION ALL: keeps all rows (faster)
SELECT user_id, event_date, 'purchase' as event_type FROM purchases
UNION ALL
SELECT user_id, event_date, 'view' as event_type FROM page_views
UNION ALL
SELECT user_id, event_date, 'click' as event_type FROM clicks;
-- Creates complete user event log
-- No duplicates expected (different event types)
-- UNION ALL is 3-10x faster for large tables

-- Data Science use case: combining train + validation data
SELECT * FROM train_data
UNION ALL
SELECT * FROM validation_data;
```

■ *Interview Tip: Say 'I almost always use UNION ALL in production because it's significantly faster. I only use UNION when deduplication is actually needed. Unnecessary deduplication on millions of rows wastes significant compute.' Shows performance-first thinking.*

JOINs & Relationships

Q5 Explain all types of JOINs with real Data Science examples.

Answer:

Five JOIN types every data scientist must know: (1) **INNER JOIN** — only rows matching in BOTH tables. Most common. (2) **LEFT JOIN** — all rows from left table + matching from right. Non-matching right rows = NULL. Most useful for data science (keep all users, add optional attributes). (3) **RIGHT JOIN** — all rows from right table + matching from left. Rarely used (just flip LEFT JOIN). (4) **FULL OUTER JOIN** — all rows from BOTH tables. Non-matching rows = NULL on missing side. Good for finding gaps in both datasets. (5) **CROSS JOIN** — every row from left with every row from right (Cartesian product). Use carefully — 1000 x 1000 = 1,000,000 rows!

■ **SQL Example:**

```
Tables: users (user_id, name) and orders (order_id, user_id, amount)

-- INNER JOIN: only users who have orders
SELECT u.name, COUNT(o.order_id) as order_count
FROM users u
INNER JOIN orders o ON u.user_id = o.user_id
GROUP BY u.name;

-- LEFT JOIN: ALL users, even those without orders (churned users!)
SELECT u.name,
COUNT(o.order_id) as order_count,
COALESCE(SUM(o.amount), 0) as total_spent
FROM users u
LEFT JOIN orders o ON u.user_id = o.user_id
GROUP BY u.name;
-- Users with no orders show: order_count=0, total_spent=0

-- FULL OUTER JOIN: find users without orders AND orders without users
SELECT u.user_id, o.order_id
FROM users u
FULL OUTER JOIN orders o ON u.user_id = o.user_id
WHERE u.user_id IS NULL OR o.user_id IS NULL;
-- Reveals data quality issues: orphaned orders!
```

■ *Interview Tip: Explain LEFT JOIN with a business scenario. 'For churn analysis, I always use LEFT JOIN — I start with ALL users and left join their purchase history. Users with NULL purchase dates after the join are my churned users.' Business application shows you think like a data scientist, not just a SQL writer.*

Q6 How do you find duplicate records in a table using SQL?

Answer:

Finding duplicates is a very common data cleaning task. Three approaches: (1) **GROUP BY + HAVING COUNT > 1** — find which values are duplicated. (2) **Window function ROW_NUMBER** — number duplicate occurrences, then filter for row > 1. (3) **Self-join** — join table to itself on duplicate criteria. The GROUP BY approach shows you WHAT is duplicated. The ROW_NUMBER approach lets you SELECT or DELETE the actual duplicate rows while keeping one copy.

■ **SQL Example:**

```sql
-- Table: customers (customer_id, email, name, signup_date)

-- Method 1: Find which emails are duplicated
SELECT email, COUNT(*) as occurrences
FROM customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY occurrences DESC;

-- Method 2: See ALL duplicate rows (with ROW_NUMBER)
WITH ranked AS (
SELECT *,
ROW_NUMBER() OVER (PARTITION BY email ORDER BY signup_date) as rn
FROM customers
)
SELECT * FROM ranked WHERE rn > 1;
-- rn=1: keep (first occurrence)
-- rn>1: these are duplicates

-- Method 3: DELETE duplicates, keep the earliest record
WITH ranked AS (
SELECT customer_id,
ROW_NUMBER() OVER (PARTITION BY email ORDER BY signup_date) as rn
FROM customers
)
DELETE FROM customers
WHERE customer_id IN (
SELECT customer_id FROM ranked WHERE rn > 1
);
```

■ *Interview Tip: Show the ROW_NUMBER approach — it's the most powerful because you can both identify and remove duplicates in one pattern. Say 'I use ROW_NUMBER() OVER (PARTITION BY duplicate_key ORDER BY priority) to identify duplicates, then delete where row_number > 1.'*

Q7 What is a self-join and when do you use it in Data Science?

Answer:

A **self-join** joins a table to itself. Useful when the data has hierarchical or sequential relationships within the same table. Common use cases: (1) **Employee-manager hierarchy** — employees table has manager_id referring to another employee. (2) **Sequential comparisons** — comparing each row to the next row in time series. (3) **Finding pairs** — customers who bought the same product. (4) **Network analysis** — users who follow each other.

■ **SQL Example:**

```sql
-- Use case 1: Employee hierarchy
SELECT e.name as employee, m.name as manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
-- Same table joined to itself to get employee-manager pairs

-- Use case 2: Compare consecutive sales (month-over-month)
SELECT
curr.month,
curr.revenue,
prev.revenue as prev_revenue,
curr.revenue - prev.revenue as revenue_change,
ROUND((curr.revenue - prev.revenue) / prev.revenue * 100, 2) as pct_change
FROM monthly_sales curr
LEFT JOIN monthly_sales prev
ON curr.month = prev.month + INTERVAL '1 month';
-- Joins sales table to itself, offset by 1 month

-- Use case 3: Product co-purchase analysis
SELECT a.product_id as product1, b.product_id as product2,
COUNT(*) as times_bought_together
FROM order_items a
JOIN order_items b ON a.order_id = b.order_id AND a.product_id < b.product_id
GROUP BY a.product_id, b.product_id
HAVING COUNT(*) > 10
ORDER BY times_bought_together DESC;
```

■ *Interview Tip: The month-over-month self-join is gold in interviews. Say 'For time series comparisons like MoM or YoY growth, I use a self-join with a date offset instead of writing complex subqueries.' Shows analytical SQL thinking.*

Q8 How do you handle many-to-many relationships in SQL for Data Science?

Answer:

Many-to-many relationships exist when: one student can take many courses AND one course can have many students. Direct joining creates a Cartesian-product-like mess. Solution: use a **junction/bridge table** that sits between the two tables. Each row in the junction table represents one relationship instance. In data science, many-to-many is everywhere: users ↔ products (recommendations), articles ↔ tags (NLP), patients ↔ medications (healthcare).

■ **SQL Example:**

```
-- Tables: users, products, user_purchases (junction)
CREATE TABLE user_purchases (
user_id INT,
product_id INT,
purchase_date DATE,
quantity INT,
PRIMARY KEY (user_id, product_id, purchase_date)
);

-- Find all products purchased by users who also bought Product 101
-- (Collaborative filtering foundation!)
SELECT DISTINCT up2.product_id, p.product_name,
COUNT(DISTINCT up2.user_id) as buyers
FROM user_purchases up1
JOIN user_purchases up2 ON up1.user_id = up2.user_id
JOIN products p ON up2.product_id = p.product_id
WHERE up1.product_id = 101
AND up2.product_id != 101
GROUP BY up2.product_id, p.product_name
ORDER BY buyers DESC
LIMIT 10;
-- 'Customers who bought X also bought Y'
-- This is the SQL foundation of recommendation systems!
```

■ *Interview Tip: Connect the many-to-many SQL pattern to recommendation systems. 'The user-product junction table is the foundation of collaborative filtering. The query I use to find co-purchased products is essentially item-based collaborative filtering in SQL.' Shows you connect SQL to ML concepts.*

Aggregations & GROUP BY

Q9 How do you calculate running totals and moving averages in SQL?

Answer:

Running totals and moving averages are critical for time series analysis in data science. Use **window functions** with OVER clause: (1) **Running total** — SUM() OVER (ORDER BY date). Each row shows cumulative sum up to that point. (2) **Moving average** — AVG() OVER (ORDER BY date ROWS BETWEEN N PRECEDING AND CURRENT ROW). N-period moving average. (3) **Partitioned running total** — SUM() OVER (PARTITION BY category ORDER BY date). Running total resets for each category.

■ **SQL Example:**

```
-- Daily sales table: (date, category, revenue)

-- Running total revenue
SELECT date, revenue,
SUM(revenue) OVER (ORDER BY date) as running_total
FROM daily_sales;

-- 7-day moving average
SELECT date, revenue,
AVG(revenue) OVER (
ORDER BY date
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
) as moving_avg_7d
FROM daily_sales;

-- Running total per category (resets each category)
SELECT date, category, revenue,
SUM(revenue) OVER (
PARTITION BY category
ORDER BY date
) as category_running_total
FROM daily_sales;

-- All three together:
SELECT date, category, revenue,
SUM(revenue) OVER (ORDER BY date) as overall_cumulative,
SUM(revenue) OVER (PARTITION BY category ORDER BY date) as category_cumulative,
AVG(revenue) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) as
ma_7d
FROM daily_sales
ORDER BY date;
```

■ *Interview Tip: Moving averages are asked in almost every data analyst interview. Say 'ROWS BETWEEN 6 PRECEDING AND CURRENT ROW gives a 7-day window (6 previous + current). I use this for smoothing noisy time series before analysis or modeling.' Shows time series thinking.*

Q10 What is ROLLUP and CUBE in SQL and how do you use them for data analysis?

Answer:

ROLLUP and **CUBE** are extensions of GROUP BY that automatically generate subtotals and grand totals. **ROLLUP**: Creates hierarchical subtotals going from most detailed to most aggregated. If you GROUP BY ROLLUP(year, quarter, month), you get: year+quarter+month totals, then year+quarter totals, then year totals, then grand total. **CUBE**: Creates subtotals for ALL possible combinations of columns. More comprehensive than ROLLUP. Useful for multidimensional analysis (sales cube: by

region, by product, by time). Both are shortcuts that replace writing multiple GROUP BY queries with UNION ALL.

■ SQL Example:

```
-- Sales data: (year, region, product, revenue)

-- ROLLUP: hierarchical subtotals (year > region > product)
SELECT
COALESCE(CAST(year AS VARCHAR), 'ALL YEARS') as year,
COALESCE(region, 'ALL REGIONS') as region,
COALESCE(product, 'ALL PRODUCTS') as product,
SUM(revenue) as total_revenue
FROM sales
GROUP BY ROLLUP(year, region, product)
ORDER BY year, region, product;

-- Output:
-- 2025 | North | Widget A | $50K
-- 2025 | North | Widget B | $30K
-- 2025 | North | NULL | $80K <- region subtotal
-- 2025 | South | Widget A | $40K
-- 2025 | NULL | NULL | $160K <- year subtotal
-- NULL | NULL | NULL | $300K <- grand total

-- CUBE: ALL combinations of year, region, product
SELECT year, region, product, SUM(revenue)
FROM sales
GROUP BY CUBE(year, region, product);
-- Generates 8 combinations (2^3) of subtotals
```

■ *Interview Tip: ROLLUP and CUBE questions come up in senior data analyst interviews. Say 'I use ROLLUP to build executive dashboards that show drill-down from grand total → year → region → product in a single query instead of 4 separate UNION ALL queries.' Shows practical business application.*

Q11 How do you pivot data from rows to columns in SQL?

Answer:

Pivoting transforms rows into columns — a very common data transformation in data science. Example: converting monthly sales rows into a table with one column per month. Approaches: (1) **CASE WHEN** — most portable, works in all databases. (2) **PIVOT keyword** — available in SQL Server and some other databases. The CASE WHEN approach manually creates each column. For dynamic pivots (unknown number of columns), you need dynamic SQL or handle it in Python/pandas.

■ SQL Example:

```
-- Original data (long format):
-- user_id | month | revenue
-- 101 | Jan | 500
-- 101 | Feb | 700
-- 102 | Jan | 300

-- Pivot to wide format using CASE WHEN:
SELECT
user_id,
SUM(CASE WHEN month = 'Jan' THEN revenue ELSE 0 END) as jan_revenue,
SUM(CASE WHEN month = 'Feb' THEN revenue ELSE 0 END) as feb_revenue,
SUM(CASE WHEN month = 'Mar' THEN revenue ELSE 0 END) as mar_revenue,
SUM(CASE WHEN month = 'Apr' THEN revenue ELSE 0 END) as apr_revenue,
SUM(revenue) as total_revenue
FROM monthly_user_revenue
GROUP BY user_id;

-- Output:
-- user_id | jan_revenue | feb_revenue | mar_revenue | total
-- 101 | 500 | 700 | 0 | 1200
-- 102 | 300 | 0 | 450 | 750

-- Data Science use: create user feature matrix for ML
-- Each column = one feature (revenue per month)
-- Each row = one user (observation)
-- This is feature engineering in SQL!
```

■ *Interview Tip: Connect pivoting to ML feature engineering. 'Pivoting is how I create user feature matrices directly in SQL before feeding data to ML models. Instead of handling reshaping in Python, I pivot in SQL and pull a clean feature matrix.' Shows you think end-to-end about the ML pipeline.*

Window Functions

Q12 Explain window functions in SQL. What makes them different from GROUP BY?

Answer:

Window functions perform calculations across a set of rows that are related to the current row — WITHOUT collapsing rows like GROUP BY. The key difference: GROUP BY reduces many rows to one row per group. Window functions keep all original rows but add a new column with the aggregated/ranked value. Syntax: FUNCTION() OVER (PARTITION BY col ORDER BY col ROWS/RANGE BETWEEN ...). Three categories: (1) **Ranking functions** — ROW_NUMBER, RANK,

DENSE_RANK, NTILE. (2) **Aggregate functions** — SUM, AVG, COUNT, MIN, MAX over a window. (3) **Navigation functions** — LAG, LEAD, FIRST_VALUE, LAST_VALUE.

■ SQL Example:

```
-- GROUP BY collapses rows:
SELECT department, AVG(salary) as dept_avg
FROM employees
GROUP BY department;
-- Returns 1 row per department (original rows gone)

-- Window function keeps all rows:
SELECT employee_id, name, salary, department,
AVG(salary) OVER (PARTITION BY department) as dept_avg,
salary - AVG(salary) OVER (PARTITION BY department) as diff_from_avg,
RANK() OVER (PARTITION BY department ORDER BY salary DESC) as salary_rank
FROM employees;
-- Returns ALL employee rows
-- Each row shows: their salary + dept average + how much above/below + their
rank

-- Output:
-- emp_id | name | salary | dept | dept_avg | diff | rank
-- 1 | Alice | 90000 | Eng | 80000 | +10K | 1
-- 2 | Bob | 80000 | Eng | 80000 | 0 | 2
-- 3 | Carol | 70000 | Eng | 80000 | -10K | 3
-- 4 | Dave | 75000 | HR | 75000 | 0 | 1
```

■ *Interview Tip: The key insight: 'GROUP BY reduces rows to one per group. Window functions ADD a column to all existing rows without reducing them. This lets me compare each individual row against its group — essential for outlier detection and ranking.'*

Q13 What is the difference between ROW_NUMBER, RANK, and DENSE_RANK?

Answer:

All three assign numbers to rows based on ORDER BY, but handle ties differently: **ROW_NUMBER()** — assigns unique sequential numbers. Ties get different numbers (arbitrary order for ties). No gaps. Use for: deduplication, pagination, selecting top N per group. **RANK()** — assigns same rank for ties. Next rank skips numbers equal to tie count. Example: 1,1,3 (skipped 2). Use for: competition rankings where ties share rank. **DENSE_RANK()** — assigns same rank for ties. Next rank does NOT skip. Example: 1,1,2. Use for: medal rankings, percentile calculations where you don't want gaps.

■ SQL Example:

```
-- Scores: Alice=90, Bob=90, Carol=80, Dave=70
SELECT name, score,
ROW_NUMBER() OVER (ORDER BY score DESC) as row_num,
RANK() OVER (ORDER BY score DESC) as rank,
DENSE_RANK() OVER (ORDER BY score DESC) as dense_rank
FROM scores;

-- Output:
-- name | score | row_num | rank | dense_rank
-- Alice | 90 | 1 | 1 | 1
-- Bob | 90 | 2 | 1 | 1
-- Carol | 80 | 3 | 3 | 2 <- RANK skips 2, DENSE_RANK doesn't
-- Dave | 70 | 4 | 4 | 3

-- Practical use - Top 1 per group (deduplication):
WITH ranked AS (
SELECT *,
ROW_NUMBER() OVER (PARTITION BY customer_id
ORDER BY order_date DESC) as rn
FROM orders
)
SELECT * FROM ranked WHERE rn = 1;
-- Gets most recent order per customer
-- ROW_NUMBER ensures exactly one row per customer
```

■ *Interview Tip: This is one of the most common SQL interview questions. Say 'ROW_NUMBER for deduplication (exactly one row per group), RANK for competition-style rankings where ties share rank and create gaps, DENSE_RANK when you want ties to share rank without gaps.' Clear and memorable.*

Q14 How do you use LAG and LEAD functions for time series analysis?

Answer:

LAG() accesses the value from a previous row. **LEAD()** accesses the value from a following row. Both take: column name, offset (how many rows back/forward, default=1), default value (if no row exists). Critical for: month-over-month comparisons, detecting consecutive patterns, calculating time between events, finding changes in state.

■ **SQL Example:**

```
-- Monthly revenue table: (month, revenue)

-- Month-over-month growth using LAG:
SELECT
month,
revenue,
LAG(revenue) OVER (ORDER BY month) as prev_month_revenue,
revenue - LAG(revenue) OVER (ORDER BY month) as mom_change,
ROUND(
(revenue - LAG(revenue) OVER (ORDER BY month)) /
LAG(revenue) OVER (ORDER BY month) * 100, 2
) as mom_pct_change
FROM monthly_revenue;

-- Output:
-- month | revenue | prev_revenue | change | pct_change
-- 2026-01 | 100000 | NULL | NULL | NULL
-- 2026-02 | 120000 | 100000 | 20000 | 20.00%
-- 2026-03 | 110000 | 120000 | -10000 | -8.33%

-- LEAD: predict next action (churn signal)
SELECT user_id, login_date,
LEAD(login_date) OVER (PARTITION BY user_id ORDER BY login_date) as next_login,
DATEDIFF(
LEAD(login_date) OVER (PARTITION BY user_id ORDER BY login_date),
login_date
) as days_until_next_login
FROM user_logins;
-- Users with NULL next_login = churned (no future login)!
```

■ *Interview Tip: The churn detection with LEAD is a killer example. 'I use LEAD() to find users with no future login — WHERE LEAD(login_date) IS NULL identifies users who never returned after their last session. This is cohort-level churn analysis in SQL.'*

Q15 How do you calculate percentiles and quantiles in SQL?

Answer:

Percentile calculations are essential for data distribution analysis. Methods: (1) **NTILE(n)** — divides rows into n equal buckets. NTILE(4) creates quartiles (Q1-Q4). (2) **PERCENT_RANK()** — relative rank as percentage (0 to 1). (3) **CUME_DIST()** — cumulative distribution (fraction of rows with value ≤ current row). (4) **PERCENTILE_CONT(p)** — actual percentile value using interpolation. Standard SQL. (5) **PERCENTILE_DISC(p)** — actual percentile value from existing data. These are critical for: outlier detection, salary band analysis, customer segmentation, model performance analysis.

■ **SQL Example:**

```
-- Customer spending analysis

-- NTILE: segment customers into quartiles
SELECT customer_id, total_spend,
NTILE(4) OVER (ORDER BY total_spend) as quartile,
NTILE(10) OVER (ORDER BY total_spend) as decile,
NTILE(100) OVER (ORDER BY total_spend) as percentile
FROM customer_spending;
-- quartile=1: bottom 25%, quartile=4: top 25%

-- PERCENT_RANK: what percentile is each customer in?
SELECT customer_id, total_spend,
ROUND(PERCENT_RANK() OVER (ORDER BY total_spend) * 100, 1) as percentile_rank
FROM customer_spending;

-- PERCENTILE_CONT: find the 25th, 50th, 75th, 95th percentile values
SELECT
PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY total_spend) as p25,
PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY total_spend) as median,
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY total_spend) as p75,
PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY total_spend) as p95
FROM customer_spending;
-- Use p95 as outlier threshold for anomaly detection!
```

■ *Interview Tip: Mention NTILE for customer segmentation. 'I use NTILE(10) to create deciles for customer value segmentation — decile 10 are my top 10% customers. Then I calculate retention rate per decile to understand which customer segments churn most.' Practical ML-adjacent use case.*

Subqueries & CTEs

Q16 What are CTEs (Common Table Expressions) and when do you use them over subqueries?

Answer:

A **CTE** (WITH clause) is a named temporary result set you can reference multiple times in a query. **Subquery** is an inline query inside another query. When to use CTE over subquery: (1) **Readability** — CTE has a name and is defined at the top. Subquery is buried inline. (2) **Reusability** — CTE can be referenced multiple times in the same query. Subquery must be repeated. (3) **Recursive queries** — CTEs support recursion (hierarchies, graphs). Subqueries cannot. (4) **Debugging** — you can run just the CTE to verify intermediate results. Use subquery when: the logic is simple and used only once.

■ **SQL Example:**

```
-- Finding customers above average spend — two approaches:

-- Subquery approach (harder to read):
SELECT customer_id, total_spend
FROM customers
WHERE total_spend > (
SELECT AVG(total_spend) FROM customers
);

-- CTE approach (cleaner, especially with complex logic):
WITH customer_stats AS (
SELECT AVG(total_spend) as avg_spend,
STDDEV(total_spend) as std_spend
FROM customers
),
high_value_customers AS (
SELECT c.customer_id, c.total_spend, c.segment
FROM customers c
CROSS JOIN customer_stats cs
WHERE c.total_spend > cs.avg_spend + cs.std_spend -- 1 std dev above average
)
SELECT hvc.*,
o.order_count,
o.last_order_date
FROM high_value_customers hvc
LEFT JOIN (
SELECT customer_id, COUNT(*) as order_count, MAX(order_date) as last_order_date
FROM orders
GROUP BY customer_id
) o ON hvc.customer_id = o.customer_id;
-- customer_stats CTE referenced once, but could be used multiple times
```

■ *Interview Tip: Say 'I prefer CTEs over subqueries for any multi-step logic because I can test each step independently. I name my CTEs descriptively — filtered_users, aggregated_metrics, final_output — so the query reads like documentation.' Shows code quality thinking.*

Q17 What are correlated subqueries and when do you use them?

Answer:

A **correlated subquery** references a column from the outer query — it executes once for EACH row of the outer query. Regular subqueries execute once and return a fixed result. Correlated subqueries are powerful but potentially slow (N executions for N rows). Common uses: (1) Exists check — does this row have related records? (2) Comparison to row's own group — is this row above its department's average? (3) Latest record per group — what was the last event for each user? Modern approach: replace correlated subqueries with window functions when possible — usually 10-100x faster.

■ **SQL Example:**

```
-- Correlated subquery: employees earning above their dept average
SELECT e1.name, e1.salary, e1.department
FROM employees e1
WHERE e1.salary > (
SELECT AVG(e2.salary)
FROM employees e2
WHERE e2.department = e1.department -- references outer query's e1!
);
-- Runs inner subquery ONCE PER ROW of outer query (slow for large tables)

-- Better: window function approach (runs once)
SELECT name, salary, department
FROM (
SELECT *,
AVG(salary) OVER (PARTITION BY department) as dept_avg
FROM employees
) t
WHERE salary > dept_avg;
-- Same result, much faster!

-- EXISTS correlated subquery: customers with at least one order
SELECT c.customer_id, c.name
FROM customers c
WHERE EXISTS (
SELECT 1 FROM orders o
WHERE o.customer_id = c.customer_id
AND o.order_date > '2026-01-01'
);
-- EXISTS is faster than IN for large subqueries
```

■ *Interview Tip: Say 'Correlated subqueries are intuitive but slow — they run N times for N rows. I replace them with window functions or joins whenever possible for production queries on large tables.' Shows performance awareness that distinguishes senior from junior SQL.*

Q18 How do you write recursive CTEs for hierarchical data?

Answer:

Recursive CTEs solve problems involving hierarchical or graph-like data — org charts, category trees, bill of materials, network paths. Structure: (1) **Anchor member** — the starting point (root nodes). (2) **Recursive member** — references the CTE itself to traverse one level deeper. (3) Stop condition — when no more rows to process. Use cases: employee hierarchy (CEO → VP → Manager → IC), category trees (Electronics → Phones → Smartphones), directory structures, network traversal.

■ **SQL Example:**

```
-- Employee hierarchy: find all employees under a given manager
-- employees(id, name, manager_id, level)

WITH RECURSIVE org_hierarchy AS (
-- Anchor: start with the root manager (CEO)
SELECT id, name, manager_id, 0 as depth, name as path
FROM employees
WHERE manager_id IS NULL -- CEO has no manager

UNION ALL

-- Recursive: add one more level each time
SELECT e.id, e.name, e.manager_id,
oh.depth + 1,
oh.path || ' > ' || e.name
FROM employees e
JOIN org_hierarchy oh ON e.manager_id = oh.id
)
SELECT id, name, depth, path
FROM org_hierarchy
ORDER BY depth, name;

-- Output:
-- id | name | depth | path
-- 1 | CEO | 0 | CEO
-- 2 | VP Eng | 1 | CEO > VP Eng
-- 3 | VP Sales| 1 | CEO > VP Sales
-- 5 | Manager | 2 | CEO > VP Eng > Manager
-- 8 | Engineer| 3 | CEO > VP Eng > Manager > Engineer
```

■ *Interview Tip: Mention the UNION ALL structure of recursive CTEs: 'Anchor member (base case) UNION ALL recursive member (one step deeper). Always add a depth counter and set a max depth to prevent infinite loops on circular references in the data.'*

Data Cleaning & Transformation

Q19 How do you handle NULL values in SQL for Data Science?

Answer:

NULLs are one of the trickiest parts of SQL for data scientists. Key behaviors: (1) Any arithmetic with NULL returns NULL (NULL + 5 = NULL). (2) Any comparison with NULL returns NULL, not TRUE or FALSE (use IS NULL, not = NULL). (3) Aggregate functions IGNORE NULLs (AVG ignores NULL rows,

doesn't count as 0). (4) COUNT(*) counts NULLs, COUNT(column) ignores NULLs. Functions to handle NULLs: **COALESCE(a, b, c)** — returns first non-NULL value. **NULLIF(a, b)** — returns NULL if a=b, else returns a (prevents division by zero). **IS NULL / IS NOT NULL** — correct way to check for NULLs.

■ **SQL Example:**

```
-- NULL behaviors to know:
SELECT NULL + 5; -- NULL (arithmetic)
SELECT NULL = NULL; -- NULL, not TRUE!
SELECT NULL IS NULL; -- TRUE (correct check)

-- AVG ignores NULLs (might not be what you want):
SELECT AVG(score) FROM test_results; -- ignores NULL scores
-- vs treating NULL as 0:
SELECT AVG(COALESCE(score, 0)) FROM test_results;

-- COALESCE for default values:
SELECT customer_id,
COALESCE(phone, email, 'no_contact') as contact_method,
COALESCE(total_spend, 0) as spend -- NULL becomes 0
FROM customers;

-- NULLIF to prevent division by zero:
SELECT product_id,
total_revenue / NULLIF(total_orders, 0) as avg_order_value
-- Returns NULL instead of divide-by-zero error
FROM product_summary;

-- COUNT difference:
SELECT COUNT(*) as total_rows, -- counts ALL rows including NULLs
COUNT(email) as rows_with_email -- counts only non-NULL emails
FROM customers;
```

■ *Interview Tip: The COUNT(*) vs COUNT(column) distinction is a classic trick question. Say 'COUNT(*) counts all rows including NULLs. COUNT(column) counts only non-NULL values in that column. This matters for data quality checks — if COUNT(*) and COUNT(email) differ, you have missing emails.'*

Q20 How do you write SQL for cohort analysis in Data Science?

Answer:

Cohort analysis groups users by when they first took an action (signup, first purchase) and tracks their behavior over time. It's one of the most powerful analytical techniques for understanding retention, engagement, and revenue trends. Standard approach: (1) Find each user's cohort date (first event). (2) For each subsequent period, count how many users from the cohort are still active. (3) Calculate retention rate = active users / cohort size.

■ **SQL Example:**

```
-- E-commerce retention cohort analysis
-- Tables: users(user_id, signup_date), orders(user_id, order_date)

WITH cohort_base AS (
-- Step 1: Get each user's cohort (month of first order)
SELECT user_id,
DATE_TRUNC('month', MIN(order_date)) as cohort_month
FROM orders
GROUP BY user_id
),
user_activity AS (
-- Step 2: For each order, calculate months since cohort
SELECT o.user_id,
cb.cohort_month,
DATE_TRUNC('month', o.order_date) as order_month,
DATEDIFF('month', cb.cohort_month,
DATE_TRUNC('month', o.order_date)) as months_since_cohort
FROM orders o
JOIN cohort_base cb ON o.user_id = cb.user_id
),
cohort_size AS (
SELECT cohort_month, COUNT(DISTINCT user_id) as cohort_users
FROM cohort_base
GROUP BY cohort_month
)
SELECT
ua.cohort_month,
cs.cohort_users,
ua.months_since_cohort as month_number,
COUNT(DISTINCT ua.user_id) as active_users,
ROUND(COUNT(DISTINCT ua.user_id) * 100.0 / cs.cohort_users, 1) as retention_rate
FROM user_activity ua
JOIN cohort_size cs ON ua.cohort_month = cs.cohort_month
GROUP BY ua.cohort_month, cs.cohort_users, ua.months_since_cohort
ORDER BY ua.cohort_month, ua.months_since_cohort;
```

■ *Interview Tip: Cohort analysis is the most impressive SQL question you can master. Say 'The key is DATE_TRUNC to group by month and DATEDIFF to calculate months since cohort. The result is a retention matrix where I can see exactly which acquisition month had the best long-term retention.' Shows strategic analytical thinking.*

Q21 How do you calculate session durations and user journeys in SQL?

Answer:

Session analysis requires identifying when a user's session starts and ends, and calculating the time between events. The key challenge: defining what constitutes a 'new session' (typically a gap of >30 minutes between events). Use LAG to find the previous event, then flag new sessions when the gap exceeds the threshold. This is the foundation of funnel analysis.

■ SQL Example:

```
-- User events table: (user_id, event_time, page)

-- Step 1: Flag new sessions (gap > 30 minutes = new session)
WITH session_flags AS (
SELECT user_id, event_time, page,
LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) as prev_event,
CASE
WHEN DATEDIFF('minute',
LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time),
event_time) > 30
OR LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL
THEN 1 ELSE 0
END as is_new_session
FROM user_events
),
-- Step 2: Assign session IDs
session_ids AS (
SELECT *,
SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY event_time) as session_id
FROM session_flags
),
-- Step 3: Calculate session metrics
session_metrics AS (
SELECT user_id, session_id,
MIN(event_time) as session_start,
MAX(event_time) as session_end,
DATEDIFF('minute', MIN(event_time), MAX(event_time)) as duration_min,
COUNT(*) as events_in_session,
MIN(page) as landing_page, -- first page
MAX(page) as exit_page -- last page
FROM session_ids
GROUP BY user_id, session_id
)
SELECT * FROM session_metrics WHERE duration_min > 0
ORDER BY user_id, session_start;
```

■ *Interview Tip: The SUM(is_new_session) OVER (ORDER BY event_time) trick to create session IDs is very elegant. Say 'I use a running sum of the new_session flag as the session ID — each time a new session starts, the sum increments by 1. This groups all events in the same session together.' Very impressive pattern.*

Q22 How do you detect and handle outliers using SQL?

Answer:

SQL offers several approaches for outlier detection before ML modeling: (1) **IQR method** — calculate Q1 and Q3 using PERCENTILE_CONT, compute IQR, flag values outside Q1-1.5*IQR to Q3+1.5*IQR. (2) **Z-score method** — calculate mean and stddev, flag values where |value - mean| / stddev > 3. (3) **Percentile clipping** — cap values at 1st and 99th percentile (Winsorizing). (4) **Business rules** — domain-specific rules: age > 120 is invalid, negative order amounts are errors.

■ SQL Example:

```
-- Outlier detection using IQR method:
WITH stats AS (
SELECT
PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) as q1,
PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) as q3
FROM employees
),
bounds AS (
SELECT q1, q3,
q1 - 1.5 * (q3 - q1) as lower_bound,
q3 + 1.5 * (q3 - q1) as upper_bound
FROM stats
)
SELECT e.employee_id, e.salary,
CASE
WHEN e.salary < b.lower_bound THEN 'LOW_OUTLIER'
WHEN e.salary > b.upper_bound THEN 'HIGH_OUTLIER'
ELSE 'NORMAL'
END as outlier_flag
FROM employees e
CROSS JOIN bounds b;

-- Winsorizing (cap outliers at percentiles):
WITH percentiles AS (
SELECT
PERCENTILE_CONT(0.01) WITHIN GROUP (ORDER BY amount) as p1,
PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY amount) as p99
FROM transactions
)
SELECT transaction_id,
GREATEST(LEAST(amount, p99), p1) as amount_capped
-- Clips: anything < p1 becomes p1, > p99 becomes p99
FROM transactions
CROSS JOIN percentiles;
```

■ *Interview Tip: Mention Winsorizing by name. 'I use GREATEST(LEAST(value, p99), p1) to Winsorize values — this caps extreme outliers at the 1st and 99th percentile without removing rows. Better than deletion for ML because it preserves sample size.'*

Advanced SQL for ML & Analytics

Q23 How do you write SQL for funnel analysis?

Answer:

Funnel analysis measures how many users progress through a sequence of steps (e.g., visit → signup → add to cart → purchase). The key SQL challenge: tracking ORDERED events where each user must complete steps in sequence. Two approaches: (1) **Conditional counting** — count users who completed each step regardless of order. Simple but inaccurate for strict funnels. (2) **Strict funnel with dates** — each step must happen AFTER the previous step. More accurate for conversion analysis.

■ **SQL Example:**

```
-- E-commerce funnel: visit -> signup -> purchase
-- Events table: (user_id, event_type, event_time)

-- Simple funnel (step counts regardless of order):
SELECT
COUNT(DISTINCT CASE WHEN event_type = 'visit' THEN user_id END) as visitors,
COUNT(DISTINCT CASE WHEN event_type = 'signup' THEN user_id END) as signups,
COUNT(DISTINCT CASE WHEN event_type = 'purchase' THEN user_id END) as purchasers,
-- Conversion rates
ROUND(COUNT(DISTINCT CASE WHEN event_type = 'signup' THEN user_id END) * 100.0 /
NULLIF(COUNT(DISTINCT CASE WHEN event_type = 'visit' THEN user_id END), 0), 1) as
visit_to_signup_pct,
ROUND(COUNT(DISTINCT CASE WHEN event_type = 'purchase' THEN user_id END) * 100.0
/
NULLIF(COUNT(DISTINCT CASE WHEN event_type = 'signup' THEN user_id END), 0), 1)
as signup_to_purchase_pct
FROM events
WHERE event_time >= '2026-01-01';

-- Strict funnel (each step must follow the previous):
WITH visits AS (
SELECT user_id, MIN(event_time) as visit_time
FROM events WHERE event_type = 'visit' GROUP BY user_id
),
signups AS (
SELECT s.user_id, MIN(s.event_time) as signup_time
FROM events s
JOIN visits v ON s.user_id = v.user_id AND s.event_time > v.visit_time
WHERE s.event_type = 'signup' GROUP BY s.user_id
)
SELECT COUNT(DISTINCT v.user_id) as visitors,
COUNT(DISTINCT s.user_id) as signups
FROM visits v LEFT JOIN signups s ON v.user_id = s.user_id;
```

■ *Interview Tip: The CASE WHEN inside COUNT DISTINCT is the elegant pattern for funnel analysis. Say 'I use COUNT(DISTINCT CASE WHEN event_type = step THEN user_id END) to count unique users at each funnel stage in a single pass over the data.' Clean, efficient, impressive.*

Q24 How do you write SQL for A/B test analysis?

Answer:

A/B test analysis in SQL requires: assigning users to groups, calculating metrics per group, and determining statistical significance. Key steps: (1) Verify equal group sizes (check randomization worked). (2) Calculate conversion rates per group. (3) Calculate confidence intervals. (4) Compute statistical significance (can do in SQL or pass to Python). Also check for: sample ratio mismatch (SRM),

novelty effects, network effects, and Simpson's paradox.

■ **SQL Example:**

```
-- A/B test: new checkout flow (variant) vs old (control)
-- ab_assignments(user_id, variant, assignment_date)
-- orders(user_id, order_date, amount)

-- Basic conversion analysis:
WITH experiment_users AS (
SELECT a.user_id, a.variant,
COUNT(o.order_id) as orders_placed,
COALESCE(SUM(o.amount), 0) as total_revenue
FROM ab_assignments a
LEFT JOIN orders o
ON a.user_id = o.user_id
AND o.order_date BETWEEN a.assignment_date AND a.assignment_date + 14 -- 14-day
window
GROUP BY a.user_id, a.variant
)
SELECT
variant,
COUNT(*) as users,
SUM(CASE WHEN orders_placed > 0 THEN 1 ELSE 0 END) as converters,
ROUND(SUM(CASE WHEN orders_placed > 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2)
as conversion_rate,
ROUND(AVG(total_revenue), 2) as avg_revenue_per_user,
ROUND(SUM(total_revenue), 2) as total_revenue
FROM experiment_users
GROUP BY variant;

-- Check for Sample Ratio Mismatch:
SELECT variant, COUNT(*) as assigned_users,
COUNT(*) * 100.0 / SUM(COUNT(*)) OVER () as pct_of_total
FROM ab_assignments
GROUP BY variant;
-- Both should be ~50%. If not, randomization is broken!
```

■ *Interview Tip: Always mention Sample Ratio Mismatch. 'Before analyzing results, I always check if both groups have approximately equal sizes. If control has 60% of users instead of 50%, the randomization is broken and results are invalid.' This shows statistical rigor.*

Q25 How do you write SQL to calculate customer lifetime value (CLV)?

Answer:

Customer Lifetime Value is one of the most important metrics in data science. Two approaches: (1) **Historical CLV** — sum of all past revenue from a customer. Simple, backward-looking. (2) **Predictive**

CLV — predict future revenue. Uses purchase frequency, average order value, customer lifespan. SQL is great for historical CLV and creating features for predictive CLV models. Key metrics: average order value, purchase frequency, customer lifespan, churn probability.

■ **SQL Example:**

```
-- Historical CLV calculation:
WITH customer_metrics AS (
SELECT
customer_id,
COUNT(DISTINCT order_id) as total_orders,
SUM(amount) as total_revenue,
AVG(amount) as avg_order_value,
MIN(order_date) as first_order,
MAX(order_date) as last_order,
DATEDIFF('month', MIN(order_date), MAX(order_date)) as customer_lifespan_months,
DATEDIFF('day', MAX(order_date), CURRENT_DATE) as days_since_last_order
FROM orders
GROUP BY customer_id
)
SELECT *,
-- Historical CLV
total_revenue as historical_clv,
-- Avg monthly revenue
CASE WHEN customer_lifespan_months > 0
THEN total_revenue / customer_lifespan_months
ELSE total_revenue END as monthly_revenue,
-- Purchase frequency (orders per month)
CASE WHEN customer_lifespan_months > 0
THEN total_orders * 1.0 / customer_lifespan_months
ELSE total_orders END as order_frequency,
-- Churn flag (no order in 90 days)
CASE WHEN days_since_last_order > 90 THEN 1 ELSE 0 END as is_churned,
-- CLV segment
NTILE(5) OVER (ORDER BY total_revenue) as clv_quintile
FROM customer_metrics;
```

■ *Interview Tip: Connect CLV to ML. 'I use this SQL query to create features for a CLV prediction model — purchase frequency, avg order value, customer lifespan, and days since last order are all strong predictors. The historical CLV becomes my target variable.' Shows you think end-to-end from SQL to ML.*

Q26 How do you optimize slow SQL queries in production?

Answer:

Query optimization is a critical production skill. Step-by-step: (1) **EXPLAIN/EXPLAIN ANALYZE** — always start here. See the query execution plan and identify bottlenecks. (2) **Indexes** — ensure columns

in WHERE, JOIN ON, and ORDER BY clauses are indexed. Covering index includes all queried columns. (3) **Avoid SELECT *** — select only needed columns. Reduces data transfer and allows index-only scans. (4) **Filter early** — filter rows as early as possible to reduce data volume for subsequent operations. (5) **Avoid functions on indexed columns** — WHERE YEAR(date) = 2026 prevents index use. Use WHERE date BETWEEN '2026-01-01' AND '2026-12-31'. (6) **Partition pruning** — on partitioned tables, filter on the partition key.

■ **SQL Example:**

```
-- Slow query:
SELECT u.name, COUNT(o.order_id)
FROM users u, orders o
WHERE u.user_id = o.user_id
AND YEAR(o.order_date) = 2026 -- Prevents index use on order_date!
AND u.country = 'India'
GROUP BY u.name;

-- Optimized:
SELECT u.name, COUNT(o.order_id)
FROM users u
JOIN orders o ON u.user_id = o.user_id -- Explicit JOIN (clearer)
WHERE o.order_date >= '2026-01-01' -- Index can be used!
AND o.order_date < '2027-01-01'
AND u.country = 'India' -- Index on country helps
GROUP BY u.name;

-- Check query plan:
EXPLAIN ANALYZE
SELECT ...;
-- Look for: Sequential Scan (bad on large tables),
-- Index Scan (good), Nested Loop with many rows (bad)

-- Create covering index:
CREATE INDEX idx_orders_covering
ON orders(user_id, order_date, order_id, amount);
-- Allows index-only scan without touching main table
```

■ *Interview Tip: Mention EXPLAIN ANALYZE first. 'Before optimizing anything, I run EXPLAIN ANALYZE to see the actual execution plan. Premature optimization without profiling wastes time. The query plan tells you exactly what to fix.' Shows systematic debugging.*

Q27 How do you use SQL for feature engineering for ML models?

Answer:

SQL is an extremely powerful tool for feature engineering on large datasets — often faster than pandas for data that lives in a database. Key feature types you can create in SQL: (1) **Aggregation features** —

count, sum, avg, max per entity. (2) **Time-based features** — days since last event, rolling averages, day of week, hour of day. (3) **Ratio features** — conversion rate, fill rate, percentage of total. (4) **Lag features** — previous values using LAG(). (5) **Category encoding** — target encoding using group averages. (6) **Interaction features** — combinations of existing features.

■ **SQL Example:**

```
-- Create ML-ready feature matrix for churn prediction:
WITH user_purchase_features AS (
SELECT
user_id,
COUNT(*) as total_orders, -- F1: order count
SUM(amount) as total_revenue, -- F2: total revenue
AVG(amount) as avg_order_value, -- F3: avg order value
STDDEV(amount) as order_value_stddev, -- F4: order consistency
MAX(order_date) as last_order_date,
DATEDIFF('day', MAX(order_date), CURRENT_DATE) as recency_days, -- F5: recency
DATEDIFF('day', MIN(order_date), MAX(order_date)) as tenure_days, -- F6: tenure
COUNT(*) * 1.0 / NULLIF(DATEDIFF('month', MIN(order_date), MAX(order_date)), 0)
as order_frequency_per_month -- F7: frequency
FROM orders
GROUP BY user_id
),
user_behavior_features AS (
SELECT user_id,
SUM(CASE WHEN page_type = 'product' THEN 1 ELSE 0 END) as product_views,
SUM(CASE WHEN event_type = 'add_to_cart' THEN 1 ELSE 0 END) as cart_adds,
-- F8: cart abandonment rate
SUM(CASE WHEN event_type = 'add_to_cart' THEN 1 ELSE 0 END) * 1.0 /
NULLIF(SUM(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END), 0)
as cart_to_purchase_ratio
FROM user_events
GROUP BY user_id
)
SELECT p.*, b.product_views, b.cart_adds, b.cart_to_purchase_ratio,
CASE WHEN p.recency_days > 90 THEN 1 ELSE 0 END as is_churned -- TARGET
FROM user_purchase_features p
LEFT JOIN user_behavior_features b ON p.user_id = b.user_id;
```

■ *Interview Tip: This query creates a production-ready ML feature matrix directly in SQL. Say 'I do as much feature engineering as possible in SQL before pulling data into Python — SQL operates at the database level and is often 10-100x faster than equivalent pandas operations on large datasets.'*

Q28 How do you write SQL to analyze time series data for forecasting?

Answer:

Time series analysis in SQL focuses on creating the features that ML forecasting models need: lag features, rolling statistics, seasonality indicators. Key techniques: (1) **Lag features** — previous N values using LAG(). (2) **Rolling statistics** — moving average, rolling std, rolling min/max using window functions. (3) **Seasonality features** — day of week, month, quarter, holiday flags. (4) **Trend indicators** — linear trend, exponential growth rate. (5) **Gap filling** — some days may have no data. Generate all dates and LEFT JOIN to fill gaps.

■ SQL Example:

```
-- Daily sales time series feature engineering for ML forecasting:
WITH date_spine AS (
-- Generate all dates (no gaps)
SELECT generate_series('2025-01-01'::date, '2026-12-31'::date, '1
day'::interval)::date as date
),
sales_filled AS (
SELECT d.date,
COALESCE(s.revenue, 0) as revenue -- 0 for days with no sales
FROM date_spine d
LEFT JOIN daily_sales s ON d.date = s.sale_date
),
features AS (
SELECT
date, revenue,
-- Lag features (for autoregressive model)
LAG(revenue, 1) OVER (ORDER BY date) as revenue_lag1,
LAG(revenue, 7) OVER (ORDER BY date) as revenue_lag7, -- same day last week
LAG(revenue, 30) OVER (ORDER BY date) as revenue_lag30, -- same day last month
-- Rolling statistics
AVG(revenue) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) as
ma7,
AVG(revenue) OVER (ORDER BY date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW) as
ma30,
STDDEV(revenue) OVER (ORDER BY date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW) as
std30,
-- Seasonality features
EXTRACT(DOW FROM date) as day_of_week, -- 0=Sunday
EXTRACT(MONTH FROM date) as month,
EXTRACT(QUARTER FROM date) as quarter,
CASE WHEN EXTRACT(DOW FROM date) IN (0, 6) THEN 1 ELSE 0 END as is_weekend
FROM sales_filled
)
SELECT * FROM features WHERE date >= '2025-02-01' -- skip rows with NULL lags
ORDER BY date;
```

■ *Interview Tip: Mention the date spine pattern. 'I always create a date spine — a complete sequence of all dates — and LEFT JOIN actual data to it. Without this, days with zero sales are missing from the dataset, which breaks time series models that expect regular intervals.'*

Q29 How do you write SQL queries for geospatial and JSON data in modern data warehouses?

Answer:

Modern data warehouses (BigQuery, Snowflake, Redshift) support semi-structured and spatial data. **JSON/VARIANT handling**: extract nested fields, flatten arrays, parse event properties stored as JSON. **Geospatial**: calculate distances between lat/lng coordinates, find points within a radius, spatial joins. These are increasingly important as data comes from mobile apps (GPS), clickstream events (JSON), and APIs.

■ **SQL Example:**

```
-- JSON data: events table with properties stored as JSON
-- event_properties: {'device': 'mobile', 'country': 'IN', 'session_id': 'abc'}

-- BigQuery syntax:
SELECT
user_id,
event_type,
JSON_EXTRACT_SCALAR(event_properties, '$.device') as device,
JSON_EXTRACT_SCALAR(event_properties, '$.country') as country,
JSON_EXTRACT_SCALAR(event_properties, '$.session_id') as session_id
FROM events
WHERE JSON_EXTRACT_SCALAR(event_properties, '$.country') = 'IN';

-- Snowflake syntax:
SELECT event_properties:device::STRING as device,
event_properties:country::STRING as country
FROM events;

-- Geospatial: find stores within 10km of a user
SELECT store_id, store_name,
ST_DISTANCE(
ST_GEOGPOINT(store_longitude, store_latitude),
ST_GEOGPOINT(user_longitude, user_latitude)
) / 1000 as distance_km
FROM stores
WHERE ST_DWITHIN(
ST_GEOGPOINT(store_longitude, store_latitude),
ST_GEOGPOINT(user_longitude, user_latitude),
10000 -- 10,000 meters = 10km
)
ORDER BY distance_km;
```

■ *Interview Tip: Mention JSON flattening for ML feature extraction. 'Mobile app events often come as JSON blobs. I use JSON_EXTRACT to flatten properties into columns, then aggregate into user-level features for ML models. This is how I extract device type, country, and behavior features from raw event logs.'*

You Made It! ■

30 SQL for Data Science — Complete.

JOINs | Window Functions | CTEs | Cohort | A/B Testing | Feature Engineering

■ **github.com/amanailab/Production-level-Generative-AI-Interview-Questions**

■ 30 AI Agent | 30 LLM | 30 RAG | 30 Prompt Engineering | 30 Agentic AI

■ 30 GenAI Project | 21 Python | 21 MLOps | 21 LLMOps | 50 ML

amanailab.com

Built with ❤ by AmanAI Lab | 2026